

Niveau	Groupes	Phase
1re année cycle ingénieur	Binômes ou trinômes	Phase 5 — Interfaces

Phase 5 — Interfaces applicatives

Front-office & Back-office

CubeSat Earth Observation System — NanoOrbit

Prérequis

Rappel : Ce TP est la suite directe des Phases 1 à 4. Votre base nanoOrbit_db doit être créée, peuplée et administrée avant de commencer. Vous allez construire une application web connectée à votre propre base de données.

1. Vue d'ensemble

Ce mini-projet se déroule sur **3 séances (10h30)**. Il aboutit à une application web unique comportant deux espaces distincts : un front-office de consultation et un back-office d'administration, séparés par un système d'authentification par profil.

Séance	Contenu	Objectif
Séance 1 (3h30)	Architecture & connexion à la base	Application qui tourne et interroge nanoOrbit_db
Séance 2 (3h30)	Front-office — consultation	4 vues métier de la Phase 3 accessibles en lecture
Séance 3 (3h30)	Back-office + démo (5 min/groupe)	Actions métier par profil, démo finale

Note

Le cœur de l'évaluation reste la base de données. L'interface est le prétexte : ce qui est évalué, c'est la justesse des requêtes SQL, la cohérence avec les droits définis en Phase 4, et la bonne exploitation des vues et procédures stockées.

2. Niveaux de réalisation

Chaque groupe choisit librement son niveau en début de Séance 1 et l'indique dans l'en-tête de son code. Les niveaux déterminent l'autonomie attendue, pas les fonctionnalités obligatoires — celles-ci sont identiques pour tous.

Niveau	Profil ciblé	Stack imposée ?	Architecture
1 Découverte	Débutants en développement web	Non (Node.js + HTML ou Python/Flask)	Guidée - endpoints décrits dans le sujet

Niveau	Profil ciblé	Stack imposée ?	Architecture
2 Autonome	À l'aise avec une stack web	Non - libre	Libre - exigences fonctionnelles imposées
3 Expert	Profil DevOps / full-stack	Non - libre	Libre + contraintes supplémentaires (auth réelle, triggers, gestion d'erreurs)

Note

Si votre base de Phase 2 est incomplète ou défaillante, vous pouvez demander la base de référence au chargé de TD sans pénalité sur la Phase 5. Une note réduite sera appliquée sur les phases antérieures.

3. Choix technologique

3.1 Options validées

Toute stack permettant une connexion MySQL est acceptée aux niveaux 2 et 3. Voici les options recommandées selon votre profil :

Stack	Backend	Frontend	Connecteur MySQL	Niveau conseillé
Node.js / Express	Express.js	HTML/CSS/JS ou React	mysql2	1, 2, 3
Python / Flask	Flask	Jinja2 templates ou React	mysql-connector-python	1, 2
Python / Django	Django ORM	Django templates	mysqlclient	2, 3
Java / Spring Boot	Spring MVC	Thymeleaf ou React	MySQL Connector/J	2, 3
Next.js (full-stack)	API Routes	React	mysql2	3

3.2 Connexion à nanoOrbit_db — exemples de démarrage

Node.js / Express (Niveau Découverte)

```
// Installation
npm init -y
npm install express mysql2 express-session

// db.js - connexion à la base
const mysql = require('mysql2/promise');
const pool = mysql.createPool({
  host: 'localhost',
  user: 'opérateur_sat', // profil Phase 4
```

```
password: 'VotreMotDePasse',  
database: 'nanoOrbit_db',  
waitForConnections: true  
});  
module.exports = pool;
```

Python / Flask (alternative)

```
# Installation  
pip install flask mysql-connector-python  
  
# app.py  
import mysql.connector  
conn = mysql.connector.connect(  
    host='localhost',  
    user='opérateur_sat',  
    password='VotreMotDePasse',  
    database='nanoOrbit_db'  
)
```

4. Authentification

L'application propose un formulaire de connexion unique. Selon le profil détecté, l'utilisateur est redirigé vers l'espace front-office ou back-office. Deux approches sont proposées — choisissez celle qui correspond à votre niveau.

Option A — Authentification MySQL réelle (Niveaux 2 & 3 recommandé)

La connexion à la base est établie avec les identifiants saisis dans le formulaire. Si la connexion échoue, l'accès est refusé. Le rôle de l'utilisateur est détecté en interrogeant `information_schema`.

```
// Connexion avec les identifiants du formulaire  
const pool = mysql.createPool({  
    host: 'localhost',  
    user: req.body.username,    // ex: 'opérateur_sat'  
    password: req.body.password,  
    database: 'nanoOrbit_db'  
});  
  
// Détection du profil  
const [rows] = await pool.query(  
    "SELECT GRANTEE FROM information_schema.USER_PRIVILEGES" +  
    " WHERE GRANTEE LIKE ?"  
    , [`${req.body.username}@'localhost'`]  
);
```

Option B — Authentification applicative (Niveau Découverte)

Les comptes sont définis dans le code (ou dans un fichier de configuration). La connexion à MySQL se fait toujours avec un compte technique unique. Le rôle de l'utilisateur est géré par la session applicative.

```
// users.js - comptes applicatifs (NE PAS versionner avec de vrais mots de passe)
const USERS = {
  'opérateur_sat': { password: 'Test1234!', role: 'opérateur' },
  'analyste_data': { password: 'Test1234!', role: 'analyste' },
  'resp_mission': { password: 'Test1234!', role: 'responsable' },
  'admin_nano': { password: 'Test1234!', role: 'admin' },
};

// Après login réussi : req.session.role = USERS[username].role
```

Profil	Accès front-office	Accès back-office	Actions disponibles
analyste_data	✓ Complet	✗ Refusé	Lecture seule — toutes les vues
opérateur_sat	✓ Complet	✓ Partiel	Satellites + fenêtres de communication
resp_mission	✓ Complet	✓ Partiel	Missions + participations
admin_nano	✓ Complet	✓ Complet	Toutes les actions

5. Front-office — Espace de consultation

Le front-office est accessible à tous les profils authentifiés. Il expose les données NanoOrbit en lecture seule, en s'appuyant sur les vues créées en Phase 3. Aucune écriture ne doit être possible depuis cet espace.

5.1 Écrans obligatoires (tous niveaux)

FO-01 — Tableau de bord des satellites opérationnels

Source : VUE_SATELLITES_OPERATIONNELS

Afficher un tableau listant les satellites opérationnels avec : identifiant, nom, format, date de lancement, type d'orbite, altitude, capacité batterie. Un indicateur visuel doit distinguer les satellites selon leur format (1U / 3U / 6U / 12U).

Note

Niveau Découverte — endpoint suggéré : GET /api/satellites →
SELECT * FROM VUE_SATELLITES_OPERATIONNELS

FO-02 — Bilan des communications par satellite

Source : VUE_BILAN_COMMUNICATIONS

Afficher pour chaque satellite : nombre de fenêtres réalisées, volume total téléchargé (Mo), volume moyen par fenêtre, date de la dernière communication, nombre de stations différentes contactées. Mettre en évidence le satellite le plus actif.

Note

Niveau Découverte — endpoint suggéré : GET /api/communications →
SELECT * FROM VUE_BILAN_COMMUNICATIONS ORDER BY volume_total
DESC

FO-03 — Tableau de bord des missions

Source : VUE_TABLEAU_DE_BORD_MISSIONS

Afficher les missions actives avec : identifiant, nom, zone géographique cible, date de début, nombre de satellites participants, nombre de satellites opérationnels parmi eux. Signaler visuellement les missions sous-dotées (moins de satellites opérationnels que de participants).

Note

Niveau Découverte — endpoint suggéré : GET /api/missions → SELECT
* FROM VUE_TABLEAU_DE_BORD_MISSIONS

FO-04 — Alertes instruments

Source : VUE_ALERTES_INSTRUMENTS

Afficher la liste des instruments en état anormal (dégradé ou hors service). La colonne de priorité (CRITIQUE / SURVEILLANCE) doit être visuellement saillante — rouge pour CRITIQUE, orange pour SURVEILLANCE. Un compteur d'alertes critiques doit apparaître en haut de page.

Note

Niveau Découverte — endpoint suggéré : GET /api/alertes → SELECT
* FROM VUE_ALERTES_INSTRUMENTS ORDER BY priorite DESC

5.2 Écran optionnel (bonus)

FO-05 — Historique des fenêtres de communication

Option

Afficher l'historique des fenêtres de communication avec filtrage par satellite et par statut (Réalisée / Planifiée / Échouée). Inclure : date/heure de début, durée, satellite, station, élévation max, volume de données. Tri par date décroissante.
Source : jointure FENETRE_COM ↔ SATELLITE ↔ STATION_SOL (requête R03 de la Phase 3).

6. Back-office — Espace d'administration

Le back-office est réservé aux profils ayant des droits d'écriture (opérateur_sat, resp_mission, admin_nano). Chaque action doit être protégée : vérifier le profil connecté avant toute opération d'écriture, et afficher un message d'erreur explicite si l'action est refusée.



Toute action d'écriture doit respecter les droits définis en Phase 4. Si un profil tente une action non autorisée, l'application doit refuser clairement (message d'erreur visible) sans provoquer d'exception non gérée.

6.1 Actions obligatoires (tous niveaux)

BO-01 — Modifier le statut d'un satellite

Profils autorisés : opérateur_sat, admin_nano

Afficher la liste des satellites avec leur statut actuel. Permettre la modification du statut via un formulaire ou un menu déroulant (Opérationnel / En veille / Désorbité). Afficher un message de confirmation après la mise à jour.

Note

Niveau	Découverte	—	endpoint	suggéré	:	POST
	/api/satellites/:id/statut	→	UPDATE SATELLITE SET statut =			
	? WHERE id_satellite = ?					

BO-02 — Planifier une fenêtre de communication

Profils autorisés : opérateur_sat, admin_nano

Formulaire de création d'une fenêtre de communication avec : satellite (liste déroulante des opérationnels), station au sol (liste déroulante des actives), date/heure de début, durée (en secondes, entre 1 et 900), élévation maximale. L'insertion doit respecter la contrainte CHECK de Phase 2.

Note

Niveau	Découverte	—	endpoint	suggéré	:	POST
	/api/fenetres	→	INSERT INTO FENETRE_COM (...)	VALUES (...)		

BO-03 — Assigner un satellite à une mission

Profils autorisés : resp_mission, admin_nano

Formulaire d'assignation : choisir un satellite opérationnel, choisir une mission active (non terminée), saisir le rôle du satellite. Vérifier que la participation n'existe pas déjà avant l'insertion. Afficher un message d'erreur si la mission est terminée.

Note

Niveau Découverte — endpoint suggéré : `POST /api/participations → INSERT INTO PARTICIPATION (...) VALUES (...)`

BO-04 — Désorbiter un satellite

Profils autorisés : `admin_nano` uniquement

Action destructive : passer le statut d'un satellite à "Désorbité". Afficher une confirmation explicite avant l'action ("Êtes-vous sûr ?"). Pour les groupes ayant réalisé la Phase 4.5 : appeler la procédure stockée `desorbiter_satellite()` et afficher le nombre d'opérations effectuées (fenêtres annulées).

Note

Phase 4.5 — appel procédure : `CALL desorbiter_satellite(?, @n);`
`SELECT @n;`

Sans Phase 4.5 : `UPDATE SATELLITE SET statut = 'Désorbité'`
`WHERE id_satellite = ?`

6.2 Action optionnelle (bonus)

BO-05 — Gérer les instruments embarqués

Option

Permettre l'ajout et la suppression d'un instrument sur un satellite (table `EMBARQUEMENT`). Afficher pour chaque satellite la liste de ses instruments avec leur état de fonctionnement. Permettre la modification de l'état (Nominal / Dégradé / Hors service).

Profils autorisés : `opérateur_sat`, `admin_nano`. Respecter la contrainte `RG-I03` (instrument non simultanément sur deux satellites) — si la Phase 4.5 a été réalisée, le trigger `T03` gère automatiquement cette vérification.

7. Guide de démarrage — Séance 1 (Niveau Découverte)

Cette section est destinée aux groupes de niveau Découverte. Elle décrit pas à pas la structure du projet et les premières étapes à réaliser en Séance 1.

7.1 Structure du projet recommandée

```
nanoOrbit-app/  
├── .env                ← identifiants MySQL (ne pas versionner)  
├── .gitignore  
├── package.json  
├── server.js           ← point d'entrée Express  
├── db.js               ← connexion MySQL  
├── routes/  
│   ├── auth.js        ← login / logout  
│   ├── front.js       ← routes front-office (GET uniquement)  
│   └── back.js        ← routes back-office (GET + POST)  
├── public/  
│   ├── css/style.css  
│   └── js/main.js  
└── views/  
    ├── login.html  
    ├── front/         ← pages front-office  
    └── back/          ← pages back-office
```

7.2 Objectif fin de Séance 1

À la fin de la première séance, chaque groupe doit avoir :

1. Un projet initialisé et versionné (Git)
2. La connexion à nanoOrbit_db fonctionnelle (tester avec un SELECT 1)
3. Une page de login opérationnelle (formulaire + session)
4. Au moins une route qui retourne des données de la base (ex : liste des satellites)

7.3 Checklist de vérification avant Séance 2

#	Vérification	Commande / Test
✓	Base de données accessible	SELECT DATABASE(); → nanoOrbit_db
✓	Vues Phase 3 présentes	SHOW FULL TABLES WHERE Table_type = 'VIEW';
✓	Comptes Phase 4 créés	SELECT user, host FROM mysql.user;
✓	Serveur démarre sans erreur	node server.js → Listening on port 3000
✓	Login redirige correctement	Tester avec analyste_data et operateur_sat

8. Démo finale — Séance 3

Chaque groupe dispose de **5 minutes** en fin de Séance 3 pour présenter son application. La démo est réalisée par les étudiants eux-mêmes, sur leur propre machine, en conditions réelles (application lancée, base connectée).

8.1 Scénario de démo recommandé

Étape	Action	Durée
1	Connexion en tant qu'analyste_data — montrer le front-office complet (FO-01 à FO-04)	1 min 30
2	Déconnexion puis connexion en tant qu'opérateur_sat — montrer le back-office, modifier un statut, planifier une fenêtre	1 min 30
3	Connexion en tant qu'admin_nano — montrer une action réservée (désorbiter un satellite)	1 min
4	Questions du chargé de TD	1 min

8.2 Points évalués pendant la démo

- L'application démarre et se connecte à la base sans erreur
- Les données affichées correspondent à celles de la base (pas de données en dur dans le code)
- L'authentification fonctionne et le menu s'adapte au profil connecté
- Une action non autorisée déclenche un message d'erreur clair (pas un crash)
- Le groupe est capable d'expliquer une requête SQL ou un endpoint à la demande

9. Critères d'évaluation

9.1 Grille de notation — Phase 5 (30 points)

Critère	Détail	Points
Connexion & architecture	Connexion MySQL fonctionnelle, structure de projet cohérente, gestion des sessions	4
Authentification	Login opérationnel, redirection par profil, protection des routes back-office	4
FO-01 Satellites	Données correctes depuis VUE_SATELLITES_OPERATIONNELS, indicateur visuel format	3
FO-02 Communications	Données correctes depuis VUE_BILAN_COMMUNICATIONS, mise en évidence du + actif	3
FO-03 Missions	Données correctes depuis VUE_TABLEAU_DE_BORD_MISSIONS, signal missions sous-dotées	3
FO-04 Alertes	Données correctes depuis VUE_ALERTES_INSTRUMENTS, couleur CRITIQUE / SURVEILLANCE	3
BO-01 Statut satellite	Modification fonctionnelle, droits vérifiés, confirmation affichée	2
BO-02 Fenêtre communication	Insertion fonctionnelle, contraintes respectées (durée, statut station)	3
BO-03 Assignment mission	Insertion fonctionnelle, vérification mission non terminée, doublon géré	3
BO-04 Désorbitation	Action fonctionnelle, confirmation requise, droits admin_nano vérifiés	2
Démo (5 min)	Fluidité, scénario complet, capacité à répondre aux questions techniques	4
Bonus FO-05	Fenêtres avec filtre (+ 1 point)	+1
Bonus BO-05	Gestion instruments (+ 2 points)	+2
TOTAL		30

9.2 Majoration par niveau

Niveau choisi	Exigences supplémentaires évaluées	Points supplémentaires
Niveau 1 — Découverte	Aucune — la grille ci-dessus s'applique intégralement	0
Niveau 2 — Autonome	Qualité de l'architecture (séparation des responsabilités, lisibilité du code)	+2

Niveau choisi	Exigences supplémentaires évaluées	Points supplémentaires
Niveau 3 — Expert	Auth MySQL réelle + gestion des erreurs triggers + appel procédures stockées	+4

10. Modalités de rendu

Modalité	Détail
Dépôt du code	Repository Git (GitHub / GitLab) — lien déposé sur Moodle avant la Séance 3
Nom du repository	GROUPE_NomA_NomB[_NomC]_CubeSat_Phase5
Fichier README	Instructions d'installation, variables d'environnement requises, comment lancer l'application
Fichier .env.example	Exemple de fichier .env avec les clés requises (sans valeurs réelles)
Niveau déclaré	Indiquer le niveau choisi (Découverte / Autonome / Expert) dans le README
Base de données	Script SQL de réinitialisation optionnel (dump ou script Phase 2) fourni dans /sql/
Démo	Séance 3 — 5 minutes par groupe — application lancée sur la machine du groupe

Note

Bonne pratique : Commitez régulièrement et structurez vos commits par fonctionnalité (ex : "feat: FO-01 tableau satellites", "feat: BO-02 planifier fenetre"). Un historique Git propre est un indicateur de qualité évalué au niveau 2 et 3.